

TP1 MLP with Pytorch on Fashion-MNIST

1. Part 1 - Baseline Model

Architecture MLP 784 → 256 (ReLU) → 128 (ReLU) → 10 logits, so **235 146 parameters** ReLU is used because its derivative is 1 on the active side, so gradients do not shrink layer after layer as with sigmoid, The output layer has no activation: `CrossEntropyLoss` applies log-softmax internally, so adding a softmax in `forward()` would be a "double softmax" bug (CM2).

Loss Function `nn.CrossEntropyLoss`. This is a 10-class, single-label problem, so the model must output a probability distribution (softmax) aux 10 logits puis calcule la log-vraisemblance négative de la bonne classe. MSE is designed for regression: on softmax outputs its gradient becomes very small when the model is confidently wrong.

Training configuration. Adam, $lr = 1e-3$ (the "practical first choice" of CM2), 15 epochs. batch size of 64, split 48k train, 12k val, 10k test.

Results (figure "Baseline - Adam $lr=1e-3$, No Regularization"). Final epoch: train 93.12 %, val 88.79 %, **gap 4.33 points**, test 88.21 %. The validation loss reaches its minimum at epoch 8 (0.317) and then rises to 0.360 at epoch 15, while the training loss keeps decreasing (0.241 → 0.180). The gap plot shows the gap is negative at epoch 1 (train accuracy is averaged during the epoch while the model is still improving) and then grows steadily. Both signs match the overfitting pattern of CM2 ("Overfitting & Underfitting"): after approx 8 epochs the model memorises training-specific details instead of learning features that generalise. Early stopping around epoch 8–9 would have given the same val accuracy with a lower val loss.

2. Experiment 1 : Model Architecture

Config	Train (ep15)	Val (ep15)	Gap	Test
Micro [4]	83,02 %	82,00 %	1,02	81,27 %
Small [64]	92,33 %	88,58 %	3,75	88,10 %
Baseline [256, 128]	94,06 %	89,24 %	4,82	88,63 %
Large [512, 256]	94,49 %	89,31 %	5,18	88,75 %

- **Underfitting.** Micro [4] stalls at 83 % train accuracy from epoch 10 onward ("Exp. 1 - Train Accuracy"). Its gap is the smallest (1.02), but a small gap does **not** mean good generalisation: train and val are *both* low. A 4-neuron bottleneck cannot represent the 10 classes (high bias, CM2 "Underfitting").
- **Overfitting.** Small, Baseline and Large all start with a gap approx 0 (or negative) around epochs 1–3, then the gap widens almost continuously ("Exp. 1 - Train–Validation Accuracy Gap"): train accuracy keeps rising, val accuracy flattens.
- **Convergence speed.** Larger models learn slightly faster early on (epoch-1 train acc: 65.4 % → 82.1 % → 82.3 % → 82.7 %). The big jump is from [4] to [64]. Above that, extra capacity barely changes early speed.
- **Val plateau.** Val accuracy plateaus from [64]/[256,128] onward: Large only gains +0.07 points over Baseline for 300k more parameters. Across our runs the same configuration varies by ± 0.3 points (e.g. Adam $1e-3$ [256,128] ends at 89.24 %, 89.00 % and 89.03 % at epoch 15 in three runs), so this gain is within noise. With this setup, the limit is not capacity: an MLP on flattened pixels ignores spatial structure (motivation for CNNs, CM3).

- **Bias–variance.** Final gaps $1.02 < 3.75 < 4.82 < 5.18$: the gap increases monotonically with capacity, as expected. Bias decreases with the increase of train accu and variance increases with the increase of the gap
- **Choice: [256,128].** It has the best val accuracy/size trade-off (same val as Large within noise, with half the parameters). Its overfitting (gap 4.82) is handled with regularisation in Experiment 3, as advised in CM2: keep capacity, then regularise.

3. Experiment 2A : Learning rate (SGD)

LR	Val (ep15)	Test	
1e-5	18,36 %	18,82 %	
1e-3	81,42 %	80,30 %	
0,1	89,11 %	88,50 %	

- **lr = 1e-5.** The update is so small that after 15 epochs the model is still close to random (10 %). Accuracy rises slowly and linearly (4 % -> 18 %) and the train loss curve is almost flat ("2.A - Train Loss"). This is the "too low" case of CM2
- **lr = 1e-3.** The model learns but slowly: it needs 12 epochs to reach 80 % val and is still improving at epoch 15. The train–val gap stays approx 0 because the model is still underfitting
- **lr = 0.1.** 85 % val after 1 epoch, 89 % at the end, and the gap now grows like the Adam baseline (93.77 % vs 89.11 %).
- **Cost of a bad LR.** The only change between 1e-3 and 0.1 is the learning rate, and the difference is approx 8 points of val accuracy for the same budget. With plain SGD, the LR must be searched carefully, which is expensive in practice. This motivates adaptive optimisers.

4. Experiment 2B : Optimiser Comparison

Configuration	LR	Meilleure val	Epoch de la meilleure val
SGD (meilleur)	0,1	88,96 %	10
Adam (meilleur)	1e-3	89,18 %	11

- **Best LR.** SGD is best at 0.1, Adam at 1e-3. Adam at 0.1 **diverges**: accuracy falls to 10 % (constant prediction) from epoch 3. This is the "too high" case of CM2: steps are too large, weights blow up and ReLU units likely die.
- **Speed at the best LR.** Adam reaches 85 % faster (epoch 1 vs 2), but tuned SGD reaches 88 % first (epoch 4 vs 6), and both finish at 89 % (difference 0.2 points, within run noise). So Adam does **not** converge clearly faster here. Its real advantage is **robustness**: it works well with its default LR and without any search.
- **Same LR (1e-3).** SGD reaches 81.67 % vs 89.00 % for Adam. The same number does not mean the same step: for SGD the step is (small when gradients are small), for Adam it is (normalised to order 1). This also explains why Adam still reaches 84.75 % at lr=1e-5 while SGD only reaches 18.36 %, and why Adam diverges at 0.1 where SGD is optimal. **A learning rate is only meaningful for a given optimiser** and must be tuned per optimiser.

5. Experiment 3 : Regularisation

Configuration	Train (ep20)	Val (ep20)	Gap	Test acc	Epoch of best val acc
No regularisation	95.28 %	89.34 %	5.94	88.95 %	14
Dropout 0.3	90.56 %	89.32 %	1.24	88.63 %	18
Weight decay 1e-3	90.62 %	88.50 %	2.12	87.94 %	12
Dropout + BN	91.59 %	89.36 %	2.23	89.20 %	19

- **Mechanisms.** *Dropout* randomly zeroes 30 % of hidden units at each step, so the network cannot rely on a single path and must learn redundant features. It acts like training an ensemble of sub-networks (CM2). *Weight decay* adds penalty to the loss (L2, like Ridge): large weights are penalised, which favours smoother functions with simpler decision boundaries (CM2 "Weight Decay").
- **Effect on the gap** ("*Exp. 3 - Train–Validation Accuracy Gap*"). Without regularisation the gap reaches 5.94 at epoch 20 and keeps growing. **Dropout** gives the smallest gap (1.24) with no loss in val accuracy. Part of this reduction is because train accuracy is measured in `model.train()` mode, with dropout active, while val uses `model.eval()`. This is why dropout's val is even *above* train in early epochs. **Weight decay 1e-3** reduces the gap (2.12) but also lowers val (88.50 %) and makes it unstable (87.4–89.1 % after epoch 10): with Adam lr 1e-3 seems too strong, pushing the model towards underfitting. **Dropout + BN** gives the best val (89.94 % at epoch 19) and best test (89.20 %). BatchNorm normalises activations and stabilises optimisation (CM2 "Batch Normalization"), which compensates for the slower learning caused by dropout.
- **Epoch of best val.** Regularisation delays the best epoch (18–19 vs 14 without). The regularised models fit the training set more slowly, and their val curves are still rising at epoch 20 ("*Exp. 3 - Val Accuracy*"), while the unregularised model's val stopped improving around epoch 10–14. Regularisation trades speed for generalisation: these models would benefit from more epochs combined with early stopping on val loss (CM2 "Early Stopping"). The weight-decay case (epoch 12) is an exception, explained by its oscillating val curve.

Conclusion. The final recommended configuration is **[256,128] + Dropout 0.3 + BatchNorm, Adam lr = 1e-3**, trained longer with early stopping. Architecture sets capacity (Exp. 1), the optimiser makes training efficient and robust (Exp. 2), and regularisation controls the variance created by capacity (Exp. 3). All three are part of the iterative CM2 pipeline. The remaining 89–90 % ceiling is mainly a limit of the MLP itself (flattened pixels, no spatial prior), which CNNs address.